
Tokens, Not Steps: A Failed Batch Sweep Breaks a Confound That Three Prior Blocks Could Not See

Claude Opus 5 (deep-analysis author), OPHIS¹

Abstract

We report a preregistered prediction that failed, and argue the failure is worth more than the win it was chasing. Papers 015 and 016 established that on a wall-clock-budgeted frame a mechanism’s value is inseparable from its kernel, and explained eight consecutive lever regressions with a single structural claim: every loser purchased quality with optimizer steps and lost that trade under a fixed time budget. Acting on that claim we preregistered a batch-size ladder predicting an optimum at $B \in [48, 64]$ with $\text{val_bpb} \approx 0.921\text{--}0.925$, together with a throughput model $t_{\text{step}} = F + cB$ and an explicit falsifier. The throughput model survived precisely: refit as $t_{\text{step}} = 15.0 + 1.875B$ ms it predicts every uncontended arm to within 0.3–3.6% against a 15% falsification threshold. The quality prediction failed completely. The measured curve is an inverted-U whose minimum lies at the incumbent $B=72$ ($\text{val_bpb} = 0.928257$); $B=48$ is worse by +0.0033 and +0.0060 on two seeds (2/2 same sign) and $B=32$ by +0.0109 and +0.0087, despite $B=32$ taking 3897 optimizer steps against the incumbent’s 2004. Nearly doubling the step count made the model worse. We show the step-count framing was not wrong so much as underdetermined: every prior block held batch size constant, under which steps and tokens are exactly proportional and the two hypotheses are observationally identical. Varying batch breaks the degeneracy, and the data select tokens: val_bpb tracks total tokens processed in perfect rank order across the ladder, with $\text{val_bpb} \approx a - b \ln(\text{tokens})$, $b \approx 0.06$ (pairwise range 0.046–0.086). We then cor-

rect ourselves twice, and report both corrections because the second retracts the first. Our initial reading — that $B=72$ is optimal because it is the MFU peak — rested on a contaminated arm, and the fitted model $\text{tok/s} = 2048B/(15.0 + 1.875B)$ is monotonically increasing in B , predicting that $B=288$ would clear the gate on throughput alone. We therefore tested $B=96$ at paired $n=5$. It is a null: mean -0.000406 , sd 0.002473 , 2/5 same sign, $t = -0.37$. The upward correction is thus falsified in turn, and the original conclusion survives on new evidence rather than the old reasoning. The decisive observation is that $B=96$ processes more tokens in all five pairs (+3.5% to +9.0%) while its residual against the token-law prediction is positive in all five (+0.0008 to +0.0048): the extra tokens arrive and the quality does not. The token law is therefore scoped to fixed batch, where steps are proportional to tokens — precisely the regime of the n-gram sweep in which it validated to 0.0016. Across batch changes the step-count term re-separates and cancels the token gain, leaving an interior optimum near $B=72$: below it tokens fall and dominate, above it optimizer steps fall and dominate. We restate the eight prior regressions under the corrected law, re-explain the v21 configuration regression as a token loss rather than a step loss, and derive a falsifiable hardware prediction: on B200 this frame should gain essentially nothing, because its throughput advantage converts into epochs a frozen ten-shard corpus cannot supply.

1. Introduction

A research program that only records its wins learns slowly. This paper reports a block in which the headline experiment returned no improvement at all, and in which the null nevertheless overturned an inferential frame that three prior papers had been reasoning

¹Autoresearch reimplement campaign. Correspondence to: Claude Opus 5 (OPHIS) <baiyuzhu@mit.edu>.

inside.

The setting is the `walltime_5min_h200` frame: a fixed 300s budget on one H200, `val_bpb` measured on a held-out shard, training shards frozen. Under a wall-clock budget every mechanism has two effects — what it does to the model, and what it does to the step rate — and the frame scores only their product. Paper 015 named this inseparability; paper 016 exploited it, adopting intra-document masking once expressed as FlashAttention-3 `cu_seqlens`.

Paper 016 also advanced an explanatory claim about failure. Eight levers had been tested against the adopted baseline and all eight regressed, each losing 80–150 optimizer steps, while the two historical winners both gained steps. The natural reading — which we adopted — was that optimizer steps are the currency of this frame. This paper reports that we tested that reading directly, and it did not survive.

2. The Preregistered Experiment

We registered, before launching, a throughput model and a quality prediction with falsifiers attached to each.

Throughput. $t_{\text{step}} = F + cB$, with F the batch-independent cost (dense n-gram optimizer state traversed every step, bandwidth-bound) and cB the batch-proportional compute. Initial estimate $F = 21.0$ ms, $c = 1.845$. Falsifier: measured step counts more than 15% below prediction at $B=48$ or $B=32$ refute the affine decomposition.

Quality. Optimum at $B \in [48, 64]$, `val_bpb` ≈ 0.921 – 0.925 . Gate: the best non-control arm must beat the incumbent by a paired mean ≤ -0.004072 with all seeds same-sign.

Six arms were launched concurrently on separate GPUs of a shared 8×H200 box: $B \in \{96, 72, 48, 32\}$ at seed 42, with $B \in \{48, 32\}$ replicated at seed 43. All arms held the adopted configuration fixed (fa3 varlen doc-masking, `WINDOW_PATTERN=TTTL`, `MATRIX_LR=0.04`) and set `TOTAL_BATCH_SIZE = 2048B` so that gradient accumulation was exactly one step throughout.

3. Results

The throughput model survived. Refitting on the two cleanest points gives $F = 15.0$ ms, $c = 1.875$. Against this, measured step times land at -0.3% ($B=72$), -0.2% ($B=48$), $+1.3\%$ and $+1.5\%$ ($B=32$), and $+3.6\%$ ($B=48$, seed 43) — every uncontended arm

Table 1. Batch ladder. Tokens = steps $\times$ 2048*B*. The $B=96$ arm is quarantined (see §3.1).

<i>B</i>	steps	val_bpb	MFU	t_{step}	tokens
96 [†]	1371	0.939223	17.58	278.5	269.5M
72	2004	0.928257	19.32	149.6	295.5M
48	2839	0.931542	18.27	104.8	279.1M
32	3897	0.939130	16.74	76.0	255.4M
48 _{s43}	2770	0.934343	17.83	108.8	272.3M
32 _{s43}	3904	0.937176	16.77	76.1	255.9M

inside 3.6% where the falsifier threshold was 15%. Step time in this frame is affine in batch to high accuracy, and the batch-independent term is small: 15 ms, or 7.7% of the step at $B=96$.

The quality prediction failed. No arm beat the incumbent. The curve is an inverted-U with its minimum at $B=72$, the configuration already in use. $B=48$ lost on both seeds and $B=32$ on both seeds, 4/4 same sign against the prediction. The gate was not approached, let alone cleared.

The failure is not marginal but directional. $B=32$ took 3897 optimizer steps — $1.94\times$ the incumbent — and scored $+0.0109$ worse. Under the step-count hypothesis this is close to impossible; nearly doubling the number of updates should not degrade a model that is nowhere near convergence.

3.1. A Contaminated Arm, Detected and Quarantined

The $B=96$ arm is excluded from all fits. Its step time read 195 ms at step 768 — within 1.6% of prediction — but the run finished reporting 278.5 ms with MFU decaying from 19.9% to 17.58%. That signature (on-model early, degrading late, MFU sagging) is a co-tenant arriving mid-run on the shared box, and matches the quarantine precedent set earlier in this campaign. We retain the arm’s direction (worse than $B=72$) and discard its magnitude. We record this because the arm’s exclusion was decided by a mechanical signature, not by whether it agreed with our hypothesis — it happened to agree.

4. Tokens, Not Steps

Ranking the four uncontended points by total tokens processed reproduces their `val_bpb` ranking exactly:

tokens	295.5M	279.1M	255.4M
val_bpb	0.928257	0.931542	0.939130

A logarithmic law $\text{val_bpb} \approx a - b \ln(\text{tokens})$ fits with $b \approx 0.06$. We deliberately do not quote a single fit-

ted value: pairwise slope estimates are 0.0575, 0.0855, 0.0746 (seed 42) and 0.0456 (seed 43), so the honest statement is $b \in [0.046, 0.086]$ from four estimates across two seed series. A least-squares fit to the seed-42 series alone returns 0.0755, which is false precision given that spread.

The mechanism is then immediate for the downward direction. Tokens = steps $\times$ batch, and MFU falls monotonically below $B=72$ (19.32% vs 18.27% at $B=48$ and 16.74% at $B=32$). Shrinking the batch buys optimizer steps but surrenders hardware efficiency, and the efficiency loss dominates. Smaller batches are not merely a worse trade — they process strictly fewer tokens per wall-clock second.

4.1. A Correction We Owe the Reader

We initially concluded from the above that $B=72$ is the MFU peak and therefore optimal. That conclusion does not survive its own model. Substituting the fitted affine law,

$$\text{tok/s}(B) = \frac{2048B}{15.0 + 1.875B},$$

which is monotonically increasing in B and asymptotes to 2048/1.875. Relative to $B=72$ it predicts +2.56% at $B=96$, +5.26% at $B=144$, and +8.11% at $B=288$ — the last of which clears the gate on throughput alone. The ladder cannot contradict this, because its only arm above $B=72$ was the contaminated one, quarantined in §3.1. Worse, that arm’s uncontended mid-run MFU was 19.9%, higher than $B=72$ ’s: we excluded the arm’s magnitude as contaminated and then used its final MFU anyway to crown $B=72$. The honest statement is that the ladder established $B < 72$ is worse and said nothing about $B > 72$. The upward direction was then tested as E1.

4.2. The Correction Was Also Wrong

$B=96$ against $B=72$, paired, seeds {42, 43, 44, 45, 46}, treatment and control interleaved across GPUs so co-tenant load could not align with an arm:

{+0.002735, -0.003059, +0.000757, -0.002787, +0.000325}

mean -0.000406, sd 0.002473, 2/5 same sign, $t = -0.37$. A null. The -0.003059 at seed 43, which we reported at the time as the block’s first positive signal, was a single draw from a distribution whose spread is four times the effect — the single-seed noise floor this campaign had already documented and which we nonetheless read as signal.

We have now been wrong on this lever in both directions, and the useful residue is not the batch setting

but the diagnosis. Table 1 and this experiment together show that $B=96$ delivers strictly more tokens in every pair while delivering no quality, with token-law residuals positive in all five. The law is not false; it is scoped. At fixed batch, tokens $\propto$ steps and one variable suffices — which is why the n-gram sweep, run entirely at $B=72$, was predicted to 0.0016. Change the batch and the two decouple with opposite signs, and a single-variable law must fail. The interior optimum near $B=72$ is the observable consequence: tokens bind from below, steps bind from above.

The methodological lesson is narrower than “use more seeds.” Both errors came from reading a mechanism off an arm we had ourselves flagged as unreliable — first a contaminated run, then an $n=1$ draw — while a correctly-scoped model was available in both cases.

4.3. Why Three Blocks Could Not See This

The step-count claim of paper 016 was not refuted so much as revealed to be under-determined. Every experiment supporting it held batch size fixed, and at fixed batch

$$\text{tokens} = \text{steps} \times 2048B \propto \text{steps},$$

so the two hypotheses are observationally identical — no amount of additional seeds at fixed batch could separate them. The eight regressing levers each lost 80–150 steps and the proportional tokens; the two historical winners gained both. Every prior observation is consistent with either law.

The ladder broke the degeneracy only because varying B makes steps and tokens move in opposite directions. This is a general lesson for automated campaigns: a confound invisible to replication is not defeated by more seeds, only by an experiment that decorrelates the variables. The ladder failed at its stated objective and succeeded at one it was not designed for.

4.4. Restating the Prior Blocks

Under the corrected law, the eight regressions read as token losses, and the v21 configuration re-target — previously attributed to a step-count deficit of ~ 830 steps — is re-explained as a token deficit arising from lower MFU at the `sota_swin4` constants. The distinction was immaterial to those blocks’ conclusions, which is exactly why it went unnoticed; it becomes material the moment batch size is a free variable, as it is in any capacity- or throughput-oriented lever.

5. A Hardware Prediction

The token law converts to a falsifiable claim about hardware. B200 offers $\sim 2.27\times$ H200 dense BF16 but only $1.67\times$ bandwidth. Since profiling attributes this step to small n-gram value-embedding GEMMs and $\sim 194k$ tiny fill kernels rather than large tensor-core work — and since the bandwidth-bound term F is only 7.7% of the step — we predict a realized step-rate gain of $1.3\text{--}1.6\times$, not $2.27\times$.

Applying $b \approx 0.06$, a $1.45\times$ token gain is worth ≈ -0.022 . But the frame cannot spend it. The incumbent already processes 295.5M tokens against a frozen ten-shard corpus, placing it near the two-epoch boundary; this campaign has separately measured that a $2\times$ budget reaches epoch three and $\text{val_bpb} = 0.987$, a catastrophic regression. A $1.45\times$ token increase lands at ~ 2.9 epochs, inside that failure region. We therefore predict B200 yields ≈ 0.930 (i.e. no gain) under frozen data, ≈ 0.920 if the FLOPs are reallocated to model width instead of epochs, and $\approx 0.905\text{--}0.915$ only with fresh shards. A further risk cuts the same way: FlashAttention-3 is Hopper-specific, so the adopted -0.0052 masking win is not portable to Blackwell without FA4-class kernels.

6. Next Experiments

This section is the paper’s primary output. Proposals were generated by independent multi-agent lenses (throughput; quality-per-token) and passed through adversarial critique; what the critique killed is recorded in §6.8.

6.1. The arithmetic that constrains every proposal

Two numbers determine admissibility. First, the gate is reachable on throughput alone at **+7.0%** tokens, since $0.06 \ln r = 0.004072 \Rightarrow r = 1.070$. Second, the largest intrinsic quality effect ever measured on this architecture is the n-gram capacity residual, ≈ 0.0007 . Break-even for a step-time cost s is $0.06 \cdot (-\ln(1-s))$: $1\% \rightarrow 0.00060$, $3\% \rightarrow 0.00183$, $5\% \rightarrow 0.00308$.

Consequence: intrinsic effects live at 10^{-3} and throughput effects at 10^{-2} . Any mechanism costing more than $\sim 1.5\%$ step time is dead on arrival. This single ratio explains the eight-lever losing streak better than any per-lever account, and it implies a policy: propose nothing with positive step-time cost until the negative-cost surface is exhausted.

6.2. E1 — Upward batch ladder (running)

Mechanism. $\text{tok/s} = 2048B/(15.0 + 1.875B)$ is monotonically increasing in B , asymptoting to $2048/1.875$. The 15.0ms intercept is batch-independent, so raising B dilutes it per token. Predicted: $B=96 \rightarrow +2.56\%$ ($\Delta = -0.0015$); $B=144 \rightarrow +5.26\%$ (-0.0031); $B=288 \rightarrow +8.11\%$ (-0.0047 , clears alone). Flags: `DEVICE_BATCH_SIZE`, `TOTAL_BATCH_SIZE` = $2048B$ (the code asserts divisibility; both must move together). Falsifier: measured ms/step at $B=144$ exceeding 285×1.15 refutes the affine model in the upward direction. Risk: this changes optimizer steps at fixed wall-clock, so the token law and the step-count confound re-separate here — ms/step must be reported alongside `val_bpb`. VRAM is the binding constraint above $B \approx 192$.

6.3. E2 — Regularize the memorizer instead of expanding it (running)

Mechanism. The n-gram tables are schedule-exempt: `NGRAM_WARMDOWN_RATIO` = 0.0, and the training loop excludes them from warmup and warmdown by design. Every other parameter group anneals to 0.05 of peak; the tables take 100%-rate gradient steps through the final step — that is, on second-epoch tokens. This is the maximal-memorization configuration, and the memorizer is independently known to be the sole source of this frame’s data-limitation signature. The data wall may therefore be a property of the schedule, not of the corpus. Flags: `NGRAM_VE_LR_SCALE` $\in \{0.5, 0.7, 0.85\}$, second axis `NGRAM_WD_LAMBDA` = 0.01. Cost: 0.0% — a static scale applied at group construction, changing no forward op. Break-even is therefore zero: any positive effect is profit. Falsifier: if 0.5 raises `val_bpb` by >0.002 the tables are LR-starved rather than overfit — abandon the LR axis. If all three arms sit within ± 0.0007 of baseline, late-phase table LR is not the memorization channel and E5’s premise is dead. Refinement path. The global scale is the naive version: it taxes epoch-1 learning as well as epoch-2 memorization. The mechanistically correct version applies warmdown to the n-gram groups only, decaying their LR during the second epoch alone (one line, mirroring the existing group plumbing). We expect v1 to underperform v2, and record that prediction now so a v1 loss is not misread as refutation. Novelty. A literature sweep found no work applying LR decay or weight decay to an LM memory table specifically to suppress repeated-data memorization; Engram trains its table with the opposite configuration ($5\times$ LR, zero weight decay) and does not discuss multi-epoch data.

6.4. E3 — Static-shape cu_seqlens (removes a per-step host sync)

Mechanism. The adopted doc-masking path builds boundaries with `torch.nonzero`, whose output shape is data-dependent and therefore forces a `cudaStreamSynchronize` plus D2H transfer every step, before the forward. A mandatory queue drain inside a launch-latency-bound step is the worst possible combination, and unbacked symbolic shapes independently hard-disable CUDA-graph capture. Replacing `nonzero` with `cumsum` plus a fixed-capacity scatter makes the output shape a compile-time constant, removing the sync and unblocking capture. Two further per-step syncs exist (`train_loss.item()`; the `VAL_LOSS_EVERY` probe). Predicted: recovers one drain per micro-step; +3.4% to +8.7% tokens if the build costs 5–12 ms, i.e. -0.0020 to -0.0050 . Falsifier: CUDA-event the mask build; <2 ms/step kills the proposal outright. Note: this mechanism sits inside our adopted SOTA, so the interaction is invisible to isolated testing — precisely the failure mode paper 013 identified.

6.5. E4 — Measure the step decomposition before writing any kernel

Mechanism. Three of the proposals here are gated on how the 150 ms splits. One instrumented run with `torch.cuda.Event` around (i) mask build, (ii) the n-gram hash chain, (iii) backward, (iv) optimizer step, plus per-step kernel counts, resolves them all. This has the highest expected value per GPU-minute on the list, because it kills experiments rather than running them. It also resolves a live factual dispute: the campaign has repeatedly cited “194k tiny fill kernels per step,” but at a $\sim 2\text{--}3\ \mu\text{s}$ launch floor that would be ~ 388 ms, exceeding the whole step. The figure is almost certainly a capture total ($\sim 1,940$ /step), and the prior attribution names a function that no longer exists in `train.py`.

6.6. E5 — Flagship combination: does regularization unlock stranded throughput?

Mechanism — an interaction hypothesis, not a stack. Roughly 0.020 of quality (≈ 5 gates) is currently stranded: throughput gains beyond $\sim 1.4\times$ cannot be spent because they push the run to epoch 3, where `val_bpb` collapses to 0.987. That collapse is attributable to the memorizer. Throughput levers and memorizer regularization are therefore complements: the regularizer raises the epoch ceiling, which is what converts a throughput surplus into quality. Neither half is expected to clear the gate alone. Design: 2×2 on $\{\text{throughput}\} \times \{\text{memorizer-reg}\}$, $n=5$ per

cell. The reported quantity is the interaction term $(TP+REG)-(TP)-(REG)+(\text{base})$, not any main effect. Falsifier: interaction ≤ 0.0011 at $n=5$ refutes the thesis that the epoch ceiling is memorizer-mediated — decisive either way, and worth the seeds for the negative.

6.7. E6 — Narrow the value embedding rather than shorten the table

Mechanism. The refuted lever was table rows. The other dimension is `half_kv_dim=384`. The dominant intercept cost is plausibly `embedding_dense_backward` zero-filling a $3.22\text{M} \times 384$ dense gradient per table per step ($\approx 2.5\text{ GB bf16}$), corroborated by the recorded `NGRAM_TABLE_MULT=512` $\rightarrow$ OOM. Halving the width halves that memset, the scatter, and the optimizer traffic, and frees the VRAM that E1 needs at $B=288$. Risk: HIGH on quality — this cuts the capacity of the one module known to carry the data-limitation signal. Treat as a capacity experiment, not free throughput.

6.8. Proposals the critique killed

Recording these is part of the protocol; a proposal set with no casualties means the critic was weak.

- Larger n-gram tables — refuted by measurement this block (5/5 arms regress, monotone in table size). Retired.
- `NGRAM_STATE_ROWWISE=1` — premised on reducing optimizer bandwidth; measured slower than `rowwise=0` (194.06 vs 188.48 ms). Premise falsified.
- Restoring n-gram capacity to 2^{21} — a prior critic ranked this the #2 priority, holding that v21 had “silently reverted” it. The data show the reversion was a throughput win. The critic was wrong and the campaign followed it for a block.
- `NGRAM_BACKOFF_KAPPA` — the operator-flagged candidate, rejected on mechanism: the gate is $c/(c+\kappa)$ with c accumulating monotonically, so suppression is strongest in epoch 1 and weakest in epoch 2 — backwards in time for a repeated-data failure mode. It also adds `gather/scatter_add` per hash site per layer per step, landing in the bottleneck. Break-even at 3% cost is 0.00183, i.e. $2.6\times$ the largest intrinsic effect ever measured here.
- CUDA graphs via `COMPILE_MODE=max-autotune` — attempted this block; all three arms

died silently, consistent with ~ 25 GB of graph pools on top of ~ 30 GB of n-gram tables. Blocked by memory, not only by the syncs of E3.

- `VAL_LOSS EVERY= 0` — predicted a free throughput win from removing a per-step validation pass; measured a null (1965 vs 2006 steps, i.e. slightly fewer). The probe was not on the critical path.

7. Conclusion

We preregistered a batch optimum and did not find it; the incumbent was already optimal. The throughput half of the model was confirmed to within 3.6%, and the quality half was falsified in direction, not merely in magnitude. Recording the null was necessary but not sufficient — its value came from asking why a $1.94\times$ increase in optimizer steps could make the model worse, which exposed a steps-versus-tokens degeneracy that had been structurally invisible to every prior block because all of them fixed the batch size. The frame’s currency is tokens, and tokens are maximized at the MFU peak. The immediate consequence for lever selection is that mechanisms should be judged on whether they cost token throughput, and that the most attractive remaining levers are those adding capacity without touching it — which motivates the n-gram capacity restoration launched alongside this report.